

Memory Management of LLM-Driven Storytelling for AI-Native Narrative Games

Dai-Yun Wu¹, Mu-Hong Wang², Ling-Yin Wei³, and Man-Kwan Shan⁴

^{1,2,4}Department of Computer Science,

³Bachelor Program in Artificial Intelligence Applications,
National Chengchi University, Taiwan
{111703047, 113753103, lywei, mkshan}@nccu.edu.tw

Abstract. Traditional narrative design in digital games relies heavily on manual scripting of dialogue, branching storylines, and character arcs, which scales poorly and limits replayability in open-world settings. Large Language Models (LLMs) offer new opportunities for automating dialogue, quests, and adaptive storylines, enabling AI-native narrative games where content is co-created in real time between players and AI. However, AI-native narrative games face critical challenges in memory management: finite context windows cause forgetting of player choices and events, inconsistent Non-Player Characters (NPCs) behavior, and increased latency during extended play. To address these issues, we investigate two complementary strategies for narrative memory: structured prompts for efficient short-term recall and LLM memory management for robust long-term continuity. Experiments evaluate narrative continuity, cohesion, and character arc dynamics, highlighting effectiveness of proposed approach in coherence and immersion. Moreover, to demonstrate the proposed memory mechanism, we developed Mystery Library, a Roblox-based AI-native narrative adventure game featuring six NPCs with distinct personalities who guide players through missions, dialogue, and free-form conversations.

Keywords: Large Language Models, Memory Management, Storytelling, AI-Native Narrative Game

1 Introduction

Traditional narrative design in digital games is highly labor-intensive, requiring writers and designers to develop extensive scripts of dialogue, character arcs, and branching storylines. While effective, this approach scales poorly: every new quest or interaction requires substantial human authoring effort, limiting replayability and responsiveness in open-world or player-driven environments.

The emergence of LLMs has introduced new possibilities for automating dialogue generation, adaptive story progression, and dynamic NPC behaviors [2]. An AI-native narrative game is a type of digital game where narrative content, such as dialogue, quests, story branches, and visuals, is generated dynamically by LLMs. In an AI-native narrative game, LLMs co-create with players in real time. Players influence the

unfolding story through actions, dialogue, or creative input. LLMs adapt and responds, making each playthrough unique.

However, LLM-driven games face fundamental challenges of memory management [12]. Due to finite context windows and lack of persistent memory, LLMs frequently lose track of earlier events, forget player choices, or generate contradictory NPC behavior. As narratives extend across multiple sessions and involve many NPCs, these limitations undermine coherence, immersion, and player agency. Latency further increases as prompts grow longer, making real-time gameplay impractical. These challenges highlight the need for explicit architectures that integrate LLMs with short-/long-term memory capable of preserving continuity while maintaining efficiency.

This paper investigates memory management strategies for AI-native narrative games. We proposed two complementary approaches to narrative memory—structured prompt for efficient short-term memory [15] and LLM memory mechanism for robust long-term continuity. To verify the effectiveness of the proposed approach, experiments were performed to quantify the continuity, cohesion and character arc dynamics of generated narratives. Moreover, to demonstrate the application of the proposed approach, we developed Mystery Library¹, an AI-native narrative game that integrates LLMs with memory-augmented design. As shown in Fig. 1, Mystery Library is implemented on the Roblox platform² for accessibility and rapid deployment. Roblox is an online game platform and also a game creation system that allows users to program and design games by themselves. Set in a cursed library, Mystery Library introduces six NPCs, who guide players through treasure finding missions, dialogue exchanges, and free-form conversations. Each successful exchange reveals a fragment of the overarching story, gradually uncovering the mystery behind the cursed library.



Fig. 1. Mystery Library: game screenshots.

The contributions of this paper are summarized as follows:

- We investigate the AI-native narrative game where narrative content is generated dynamically by LLMs. We present an architecture that bridges LLM techniques with digital entertainment design and provides a blueprint for future AI-native games.

¹ <https://www.youtube.com/watch?v=uZm0jRymoGM>

² <https://create.roblox.com/>

- We propose two complementary approaches for narrative memory—structured prompt for efficient short-term memory and LLM memory mechanism for robust long-term continuity—and demonstrate their integrated effectiveness.
- We develop Mystery Library, a Roblox game leveraging multi-level memory with LLMs to enable coherent, personality-consistent NPC interactions.

2 Related Work

2.1 Memory-Augmented LLMs

LLMs, while highly effective at generating fluent and contextually relevant text, remain fundamentally stateless due to their reliance on fixed-length context windows [4]. This limitation restricts their ability to sustain coherence across long-term interactions, remember past user preferences, or maintain evolving task states. To overcome these challenges, recent research has increasingly focused on augmenting LLMs with external memory mechanisms that allow information to be stored, retrieved, and updated beyond the model’s immediate context window. Such designs aim to approximate human-like memory systems by enabling persistence, selectivity, and structured knowledge organization.

Several approaches illustrate this trend. MemGPT introduced a dual-buffer architecture, separating working memory from long-term memory, coupled with scheduling policies to determine when information should be stored or recalled [11]. Mem0 proposed a minimal yet expressive operator framework {ADD, UPDATE, DELETE, NOOP}, enabling explicit and structured memory operations [4]. A-Mem extended this idea by integrating reinforcement learning to guide dynamic memory access, balancing short-term adaptability with long-term retention [16]. Memory Bank explored compositional controllers that manage lifelong agent memory across tasks, while Read-Agent highlighted retrieval-based strategies for grounding dialogue in past interactions [8]. Benchmarks such as LOCOMO further provided a systematic evaluation of agents’ ability to retrieve and reason over temporally distant conversational history. [10]

Collectively, these works mark significant progress toward building memory-augmented LLMs capable of persistent knowledge accumulation and long-term reasoning.

However, most systems remain limited to either short-term retrieval (e.g., RAG-based augmentation) or fixed operator sets without deeper integration into interactive environments. This underscores the need for approaches that unify structured memory management with personality-driven, adaptive applications—such as interactive narratives and game NPCs—where memory is central to both coherence and immersion.

2.2 Co-Creative Storytelling with Generative AI

In parallel, narrative research has explored how generative AI can support co-creative story environments. To the best of our knowledge, 1001 Nights is the only one exemplifies this approach by embedding LLMs into a collaborative storytelling game. That

1001 Nights is a narrative-driven co-creative experience where players collaborate with an AI-controlled King [12]. In 1001 Nights, gameplay is divided into two phases:

- **Storytelling Phase.** Players guide the King’s narrative so that certain keywords (e.g., “sword,” “shield”) appear naturally in the story. These words generate actual in-game weapons via GenAI systems (LLMs + Stable Diffusion).
- **Battle Phase.** Once enough weapons are collected, players use them to fight the King.

1001Night highlights the potential of AI not only as a dialogue generator but also as an interactive partner in digital entertainment. However, in 1001 Nights, the storytelling is performed by the player rather than LLM. The role of LLM is to visualize weapons. Our approach emphasizes the importance of narrative design framework where player choice, narrative flexibility, and AI adaptability converge to create immersive story experiences.

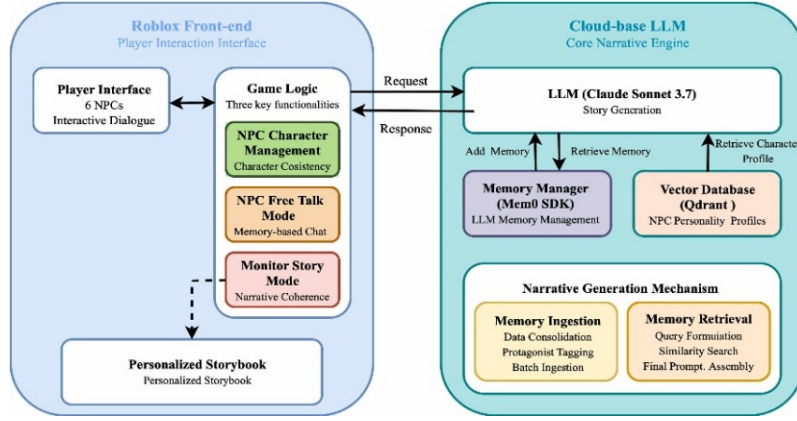


Fig. 2. System architecture diagram.

3 Proposed System Architecture

3.1 System Architecture

The proposed system architecture for AI-native narrative game is shown in Fig. 2. There are two main components: the Game Server and the LLM Server.

The Game Server is responsible for coordinating player interactions and enabling dynamic communication with multiple NPCs through three principal modules: NPC Character Management to ensure personality consistency, NPC Free Talk for memory-driven conversations beyond quest lines, and Story Monitor to preserve overall narrative coherence, along with the Personalized Storybook keeps records of player choices and past interactions.

On the LLM server side, LLM is powered by the Narrative Generation Mechanism. To consolidate, retrieve, and integrate past interactions into the story, ensuring both coherence and adaptability. Based on LLM Manager and Vector Database, two key mechanisms—Memory Ingestion and Memory Retrieval—work together to manage player interaction history and provide the model with relevant context. By combining short-term and long-term memory, the LLM produces narratives that remain consistent with past events and aligned with established NPC personalities.

In our implementation, on the LLM Server side, narrative generation is powered by the Claude Sonnet 3.7 model, supported by the mem0 SDK for multi-level memory management and Qdrant for structured NPC personality profiles. On the Game Server side, Mystery Library is developed over Roblox developer platform using LUA.

3.2 Game Server for Player Interaction

To support narrative coherence and interactive depth, the Game Server integrates three principal modules: NPC Character Management, which enforces personality consistency across interactions; NPC Free Talk, which enables memory-driven conversations extending beyond quest-specific dialogue; and the Story Monitor, which maintains global narrative coherence throughout gameplay.

- **NPC Character Management.** To ensure character consistency, each NPC is associated with a detailed personality profile. This setup allows the system to perform vector retrieval during dialogue generation, preserving the integrity of each NPC's traits and behaviors.
- **NPC Free-Talk.** Beyond quest-specific exchanges, this mode leverages LLM memory management to store and recall past conversations between players and NPCs. After completing missions, players can freely converse with NPCs, who remember prior interactions, thereby fostering a personalized and immersive experience.
- **Story Monitor.** This module safeguards narrative coherence while allowing for story diversity. By using memory management to track and retrieve key story fragments, it ensures that LLM-generated narratives remain cohesive and consistent over time.

3.3 Narrative Generation Mechanisms

Memory Ingestion. After each player interaction, the system updates its memory store through the following steps:

- *Data Consolidation.*
 - Collects player choices, dialogue exchanges, and previously generated story fragments.
 - Removes redundant or repetitive information to minimize noise and ensure efficient storage.

- Aligns story fragments into a standardized format suitable for subsequent retrieval.
- *Protagonist Tagging.*
 - Programmatically identifies the protagonist’s name (or key entities) within the narrative.
 - Tags these references with a consistent format, allowing the system to anchor continuity across interactions.
 - Ensures that future retrieval queries remain sensitive to protagonist-related events and references.
- *Batch Ingestion.*
 - Aggregates consolidated and tagged data into batches.
 - Submits these batches to the mem0 SDK’s metadata store, where semantic analysis extracts salient fragments.
 - Stores structured memory units that can be efficiently retrieved in later stages.

Memory Retrieval. Before generating each new story segment, the system executes a retrieval cycle consisting of three steps:

- *Query Formulation.*
 - Constructs a query embedding derived from the last paragraph of the ongoing narrative.
 - Encodes key thematic and contextual cues to guide retrieval toward the most relevant memory fragments.
- *Similarity Search.*
 - Performs a vector-based similarity search over the stored memory fragments.
 - Retrieves the top-k results ranked by semantic similarity scores, augmented by protagonist-aware sorting (fragments mentioning the protagonist are prioritized).
 - Outputs results as {memory, score} pairs to support transparent evaluation of relevance.
- *Prompt Assembly.*
 - Combines short-term memory (the most recent sentence of the previous paragraph) with long-term memory (retrieved fragments).
 - Integrates these elements into a structured prompt format.
 - Passes the composite prompt to the LLM, ensuring that generated narratives reflect both immediate context and accumulated history.

Together, these modules enable coherent, adaptive, and memory-augmented narrative generation. This comprehensive prompt, containing both short-term and long-term context, is then passed to the LLM for reference, enabling it to generate a more coherent and context-aware narrative.

4 EXPERIMENT

To investigate whether the integration of LLM-based memory management enhances the completeness and coherence of narrative generation compared to conventional prompt engineering, experiments were conducted using four distinct story seeds, each paired with two option sequences. To control experimental conditions, the NPC mission temperature was fixed at 0 to ensure deterministic task assignment, while the Story Monitor temperature was set to 0.4 to permit limited diversity in narrative output.

For each condition, the system generated seven consecutive and thematically connected story fragments. Beginning with the second fragment, prompts incorporated retrieved memory to simulate continuity across turns. The experiment was structured into two groups:

- **Control group.** Narrative generation relied exclusively on traditional prompt engineering techniques, without any memory augmentation.
- **Experimental group.** Narrative generation combined prompt engineering with the mem0 SDK, which actively ingested, stored, and retrieved memory fragments to inform subsequent prompts.

4.1 Coherence Analysis

This section compares story coherence between prompt-engineering only and prompt engineering with LLM memory management. Four metrics are evaluated: Perplexity, Thematic Coherence, Contradiction Rate, and Named Entity Recognition-based Coherence.

Perplexity (PPL). It is a standard metric for evaluating how well a language model predicts text, with lower values indicating greater predictability and more natural linguistic output. Originally formalized in early work on language modeling [7], PPL continues to serve as a benchmark in computational linguistics. In our implementation, we computed perplexity using an n -gram language model with Laplacian (add-one) smoothing [3]. The generated text was tokenized, partitioned into training and testing sets, and probabilities were estimated from n -grams. Perplexity was then calculated as:

$$\text{Perplexity} = 2^{-\frac{1}{M} \sum \log_2 p(w_i|h_i)} \quad (1)$$

where M is the number of test n -grams and $p(w_i|h_i)$ denotes the smoothed conditional probability of word w_i given its history h_i .

Empirical results indicate that narratives produced with LLM memory management consistently achieve lower perplexity compared to those generated with prompt engineering alone. This finding suggests that memory augmentation reduces randomness and enhances the fluency and natural flow of generated stories (see Fig. 3).

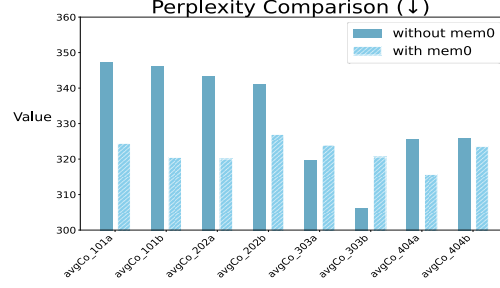


Fig. 3. Perplexity comparison.

Thematic Coherence. It assesses the consistency and continuity of themes across adjacent chapters. While classical approaches have modeled local coherence using entity grids [1], we adopt a simpler embedding-based proxy. Specifically, cosine similarity is computed between chapter embeddings (using spaCy or SBERT), with higher similarity values interpreted as stronger thematic alignment [6]. In our experiments, narratives generated with LLM memory management achieved marginally higher thematic coherence scores compared to those produced with prompt engineering alone. However, the overall differences were small. We attribute this outcome to the shared core story settings across both conditions: when the initial setup is identical, a substantial portion of the generated plot naturally converges. The key distinctions emerge in subtler aspects, such as repetitive phrasing or factual contradictions, that are difficult to capture using vector similarity alone and often necessitate human evaluation for accurate assessment (see Fig. 4).

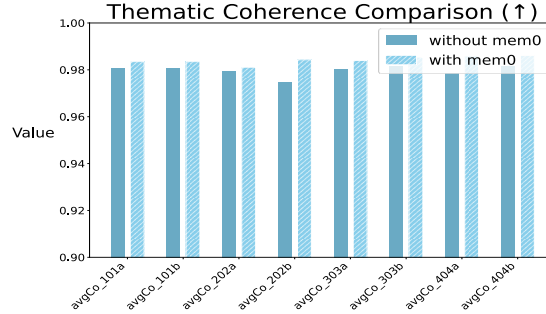


Fig. 4. Thematic coherence comparison.

Contradiction Rate. Contradiction Rate is introduced as a metric to quantify factual inconsistencies that arise across consecutive narrative segments. Maintaining coherence is particularly critical in AI-native narrative games, where LLMs generate story content dynamically and the absence of persistent authorial control increases the risk of logical conflicts. To compute this metric, we extract sentences that explicitly reference the main protagonist and pair each with a corresponding sentence from the adjacent chapter. Each sentence pair is then evaluated using the pretrained *facebook/bart*-

*large-mnli model*³, which classifies their semantic relationship as entailment, neutral, or contradiction. A contradiction is flagged when the probability of the contradiction label exceeds 0.7. By systematically identifying points where narrative continuity breaks down, Contradiction Rate provides a quantitative measure of coherence. This metric is particularly valuable for assessing whether memory-augmented narrative systems reduce factual inconsistencies compared to baseline prompt-engineered approaches, thereby improving player immersion and narrative credibility.

$$\text{Contradiction Rate} = \frac{\#\{\text{pairs with } P(\text{CONTRADICTION}) \geq 0.70\}}{\#\{\text{all evaluated pairs}\}} \quad (2)$$

Stories generated with LLM memory management consistently demonstrated a lower Contradiction rate. This indicates that the memory management approach is effective at reducing logical inconsistencies and maintaining a more coherent narrative. (Error! Reference source not found.5).

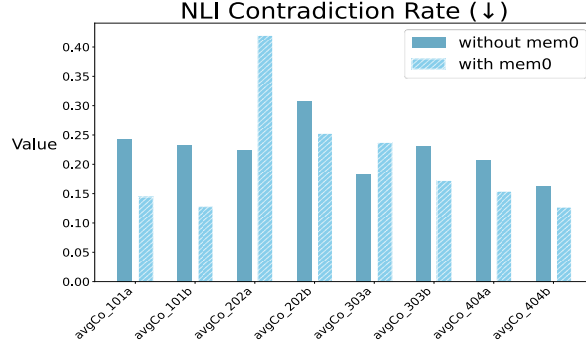


Fig. 5. NLI contradiction rate.

Named Entity Recognition (NER)-based Coherence. It evaluates the continuity of named entities, particularly person names, across narrative chapters. NER techniques identify and label mentions of entities such as persons, organizations, and locations, and have been extensively benchmarked since the CoNLL-2003 shared task [13]. For this study, we applied spaCy’s pre-trained English NER pipeline [6] to extract entities from each chapter. Based on these outputs, we define three cross chapter coherence indicators:

- **jac_median:** the median Jaccard similarity of named entity sets between adjacent chapters:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (3)$$

- **jac_low_ratio:** the proportion of adjacent chapter pairs where $J(A, B) < 0.1$, indicating low overlap.

³ <https://huggingface.co/facebook/bart-large-mnli>

- `main_persist`: the fraction of chapter boundaries where the majority protagonist (i.e., the most frequently mentioned person) remains consistent.

This analysis revealed the most pronounced differences among the evaluated coherence metrics. With LLM memory management, `jac_low_ratio` was reduced to zero across all stories, while both `jac_median` and `main_persist` were consistently higher than in the prompt-only condition. These findings demonstrate that memory augmented systems improve entity continuity and enhance character consistency throughout the narrative (see Fig. 6).

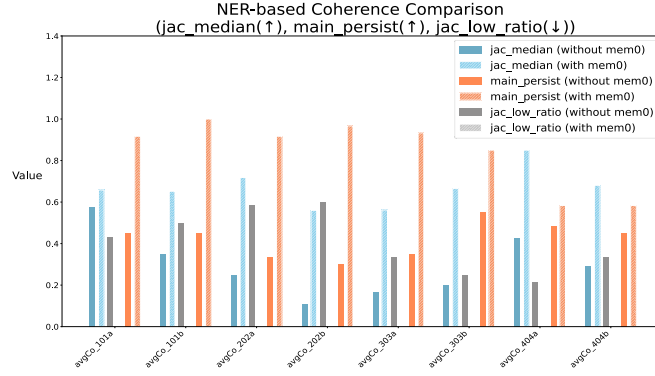


Fig. 6. NER coherence comparison.

4.2 Storyline

To evaluate narrative quality, we examine the relationships, conflicts, and long-term character development expressed within the generated stories. This analysis focuses on emotional arcs and plot progression, which together provide insight into the depth and dynamics of the narratives.

Emotions Analysis. We employed Hugging Face’s *j-hartmann/emotion-english-distil-roberta-base model*⁴, trained on six diverse datasets and capable of predicting Ekman’s six basic emotions plus a neutral class: *anger*, *disgust*, *fear*, *joy*, *neutral*, *sadness*, and *surprise* [5]. For each chapter, the model outputs probability distributions across the seven emotions. These probabilities are aggregated into an emotion matrix:

$$E = [e_{i,j}] \in \mathbb{R}^{n \times 7} \quad (4)$$

where n is the number of chapters and $e_{i,j}$ is the probability score of emotion j in chapter i . The matrix is normalized as:

$$E_{normalized} = \frac{E - \mu(E)}{\sigma(E)} \quad (5)$$

⁴ <https://huggingface.co/j-hartmann/emotion-english-distilroberta-base>

This normalization allows comparison across emotions. Each emotional sequence is then treated as a time-series signal, and we apply the ruptures library with an RBF cost to detect change points [14]. A penalty parameter of $\beta < 0.2$ was required for reliable breakpoint detection (see Fig. 7).

Without mem0. Emotional dynamics were smoother, with negative emotions like fear and sadness initially high but gradually stabilizing over time (Fig.7 (a)).

With mem0. Emotional trajectories exhibited stronger fluctuations, with pronounced peaks and troughs in both positive and negative emotions such as fear, joy, and anger (Fig. 7 (b)).

These results suggest that memory augmentation amplifies emotional responses and accelerates the resolution of negative emotions, thereby producing narratives with more dramatic and engaging character arcs. Compared to the smoother but less varied trajectories of prompt-only generation, the heightened emotional variability produced with memory management may better sustain immersion, highlight conflict and resolution cycles, and ultimately enhance the player’s sense of participating in a living, evolving story.

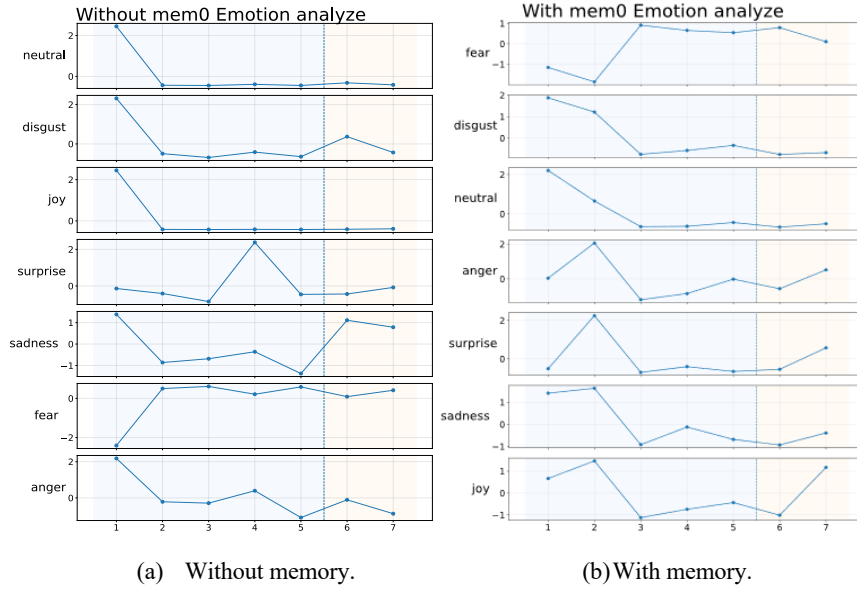


Fig. 7. Emotion analysis.

Plot Structure. It plays a critical role in shaping how readers perceive events and transitions within a narrative. To analyze structural variation, we embed each chapter using *all-MiniLM-L6-v2*⁵, a lightweight sentence-transformer for semantic similarity, and compute novelty between adjacent chapters as:

⁵ <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>

$$Novelty_i = 1 - \text{cosine_similarity}(e_i, e_{i-1}), \text{ with } Novelty_1 = 0 \quad (6)$$

where e_t and e_{i-1} are the embeddings of chapters i and $i-1$. High cosine similarity (close to 1) indicates narrative continuity and results in low novelty scores, while lower or negative similarity yields higher novelty, signaling plot shifts. To identify major structural breaks, we also applied the *ruptures* algorithm with an RBF cost, tuning the penalty parameter and setting $\beta < 0.2$ to achieve stable segmentation (Fig. 8).

Without mem0. Novelty scores display an early sharp peak followed by a gradual decline (Fig. 8 (a)). This indicates that stories become increasingly repetitive in later chapters, likely because the LLM without memory loses track of prior context and fails to maintain narrative variation.

With mem0. Novelty scores remain relatively stable across chapters, with multiple localized fluctuations (Fig. 8 (b)). This pattern suggests that memory management allows the LLM to sustain storyline diversity and introduce new plot elements, while avoiding excessive repetition.

Together, these findings suggest that memory augmentation enhances structural richness by balancing continuity with novelty, yielding narratives that are both coherent and dynamically evolving.

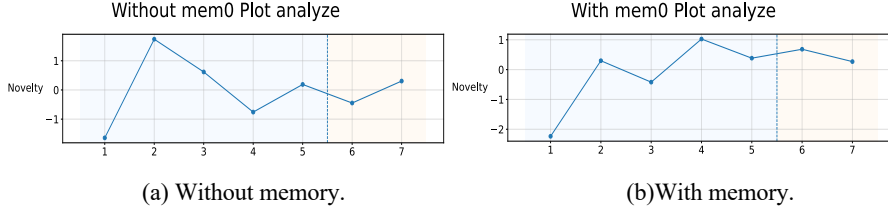


Fig. 8. Plot analysis.

4.3 Diversity Analysis

A central objective of this research is to enable players to experience multiple narrative branches within the same game. Accordingly, we compare the diversity of stories generated by the Story Monitor under two conditions: with and without LLM memory management. While the previous sections examined emotional arcs and plot progression within individual stories, this section focuses on cross-story diversity. We constructed eight parameter sets by combining four story seeds with two option sequences, fixing the generation temperature at 0.4. For each parameter set, ten stories were generated to support within-group comparisons.

Lexical Diversity. To assess lexical variation, we computed unigram term frequency distributions for each story and normalized them into probability vectors. The divergence between two stories, P and Q , was measured using Jensen–Shannon Divergence (JSD) [9]:

$$JSD(P \parallel Q) = \frac{1}{2}KL(P \parallel M) + \frac{1}{2}KL(Q \parallel M), M = \frac{1}{2}(P + Q) \quad (7)$$

where base-2 logarithms are applied. JSD is symmetric and bounded, making it well suited for cross-story comparisons. Within each parameter group, all pairwise JSD values were averaged to produce a lexical diversity index, with higher scores reflecting greater vocabulary variation.

From Fig. 9, Across the eight parameter sets (4 seeds $\times$ 2 option sequences, temperature = 0.4), the *mem0* condition exhibited equal or higher average JSD in most groups. This outcome indicates that memory management reduces redundancy and promotes lexical variety. Slightly lower JSD values in a few groups suggest concentrated vocabulary use to preserve narrative consistency. Overall, *mem0* consistently maintained or enhanced lexical across story groups, underscoring its role in supporting narrative diversity.

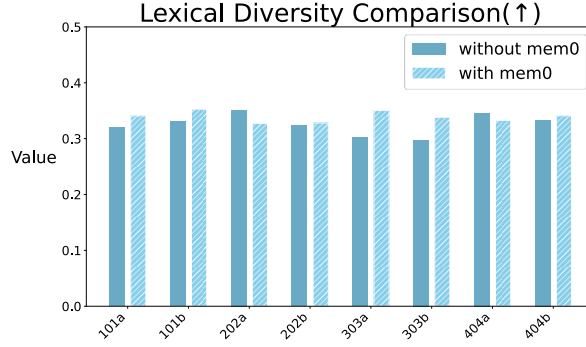


Fig. 9. Lexical diversity comparison.

Plot/Emotion Curve Diversity. To capture joint dynamics of narrative structure and emotional progression, we summarize chapter-wise emotion distributions and plot novelty into a unified diversity measure. For each story, seven-dimensional emotion scores are projected onto the first principal component (PCA) to generate an emotion curve. In parallel, plot dynamics are derived from chapter embeddings computed with a Sentence-Transformer, with novelty curves obtained from adjacent-chapter cosine similarity. After z-score standardization, the emotion and plot novelty curves are combined with equal weights (0.5/0.5) to form a composite diversity curve for each story.

Within each 10-story group, we compute all pairwise distances between diversity curves using FastDTW [17] and visualize the resulting symmetric distance matrix as a heatmap in Fig. 10.

Without mem0. The matrices appear more uniform, indicating higher homogeneity across stories and less differentiated curve shapes (Fig. 10(a)).

With mem0. In Fig. 10(b), the distribution of distances exhibits a greater dynamic range, reflected in both bright patches (indicating high distances) and dark patches (low

distances). This pattern suggests increased dispersion and more distinct structural variation among stories.

These findings demonstrate that LLM memory management fosters structured diversity in story trajectories, producing narratives that diverge meaningfully in their emotional and structural arcs while maintaining coherence. When considered alongside the lexical diversity results, these outcomes suggest that memory augmentation not only enriches the vocabulary of generated stories but also broadens their narrative and emotional trajectories, thereby enhancing replayability.

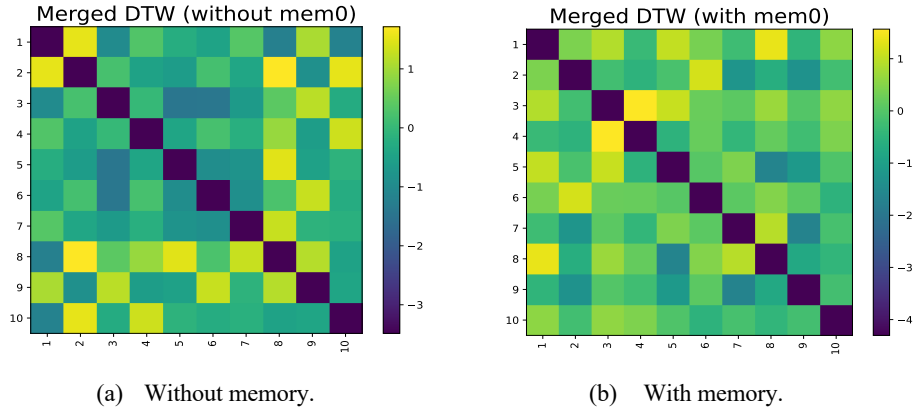


Fig. 10. DTW.

5 Conclusion

In this study, we developed Mystery Library, an AI-native narrative game designed to explore how memory-augmented LLMs can enhance interactive storytelling. By integrating structured prompts with the mem0 SDK, the system supports multi-level memory operations, allowing NPCs to sustain consistent personalities, recall past interactions, and engage in free-form conversations beyond mission-driven dialogue. Our evaluation shows that LLM memory management substantially enhances narrative quality in AI-native games across multiple dimensions. Coherence metrics show that memory integration reduces factual contradictions, strengthens thematic flow, and preserves character consistency across chapters. Storyline analysis reveals that memory augmentation amplifies emotional variability and accelerates the resolution of negative emotions, producing richer character arcs. Plot structure metrics indicate that memory helps maintain novelty across chapters, sustaining narrative variation while avoiding excessive repetition. Finally, diversity measures—both lexical and structural—highlight that mem0 increases cross-story variety, ensuring that multiple playthroughs yield distinct narrative and emotion trajectories. Together, these findings suggest that memory management is not merely a technical optimization but a fundamental design principle for AI-native narrative games, enabling stories that are more coherent, emotionally engaging, structurally dynamic, and replayable.

References

1. R. Barzilay and M. Lapata. Modeling Local Coherence: An Entity-Based Approach. *Computational Linguistics*, 34(1):1–34, (2008)
2. T. Brown, B. Mann, N. Ryder, et al. Language Models are Few-Shot Learners. *Proc. 34th Conference on Neural Information Processing Systems (NeurIPS)*, (2020)
3. S. F. Chen and J. Goodman. An Empirical Study of Smoothing Techniques for Language Modeling. *Proc. 34th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 310–318, (1996)
4. P. Chhikara, D. Khant, S. Aryan, T. Singh, D. Yadav Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv:2504.19413*, (2025)
5. P. Ekman. An argument for basic emotions. *Cognition and Emotion*, 6(3–4), 169–200 <https://doi.org/10.1080/02699939208411068>. (1992)
6. M. Honnibal, I. Montani, S. Van Landeghem, A. Boyd. spaCy: Industrial-strength Natural Language Processing in Python. *Zenodo*, DOI:10.5281/zenodo.1212303, (2020)
7. F. Jelinek, R. L. Mercer, L. R. Bahl, and J. K. Baker. Perplexity—A Measure of the Difficulty of Speech Recognition Tasks. *Program of the 94th Meeting of the Acoustical Society of America*, *Journal of the Acoustical Society of America*, 62(Suppl. 1): S63, (1977)
8. K.-H. Lee, X. Chen, H. Furuta, J. Canny, I. Fischer. A Human-Inspired Reading Agent with Gist Memory of Very Long Contexts. *Proc. 41st International Conference on Machine Learning (ICML)*, (2024)
9. J. Lin. Divergence Measures Based on the Shannon Entropy. *IEEE Transactions on Information Theory*, 37(1):145–151, (1991)
10. A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, Y. Fang. Evaluating Very Long-Term Conversational Memory of LLM Agents. *Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 13851–13870, (2024)
11. C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, J. E. Gonzalez. MemGPT: Towards LLMs as Operating Systems. *arXiv:2310.08560*, (2023)
12. Y. Sun, Z. Li, K. Fang, C. H. Lee, A. Asadipour. Language as Reality: A Co-creative Storytelling Game Experience in 1001 Nights Using Generative AI. *Proc. 19th AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, pp. 425–434, (2023)
13. E. F. Tjong Kim Sang and F. De Meulder. Introduction to the CoNLL-2003 Shared Task: Language-Independent Named Entity Recognition. *Proc. CoNLL 2003*, pp. 142–147, (2003)
14. C. Truong, L. Oudre, N. Vayatis. ruptures: change point detection in Python. *arXiv:1801.00826*, (2018)
15. J. Wei, X. Wang, D. Schuurmans, X. Wang, D. Schuurmans, M. Bosma, et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Proc. 36th Conference on Neural Information Processing Systems (NeurIPS)*, (2022)
16. W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, Y. Zhang. A-MEM: Agentic Memory for LLM Agents. *arXiv:2502.12110*, (2025)
17. H. Sakoe and S. Chiba. Dynamic Programming Algorithm Optimization for Spoken Word Recognition. *IEEE Trans. Acoustics, Speech, and Signal Processing*, (1978)